SESSION 4 · PART ONE
DATA THAT SURVIVES
FILES, CSV, JSON AND ERRORS

Information Systems Development (SE)

University of Liechtenstein

Dr. Pejman Ebrahimi

TODAY IN FIVE PARTS

01 TEXT FILES

Reading and writing with open()

02 CSV

The language of business data

03 JSON

The language of the web

04 ERRORS

try / except, and which to catch

05 THE BUILD

A catalogue that survives a restart

NEXT

Part two: serving it as an API

WHERE WE ARE



LAST WEEK

Functions, dictionaries and a list of dictionaries as a table. Flask turned that table into a web page.



TODAY

That table still vanishes when you close the program. Today it stops vanishing.

THE PROBLEM

Everything your program remembers lives in memory.

Close the program and it is gone — every product, every score, every order.

A file is how a program remembers after it stops.

01

TEXT FILES

How a program remembers
after it stops

Dr. Pejman Ebrahimi

YOUR PROGRAM FORGETS

```
basket = []
basket.append("Laptop")
basket.append("Mouse")

print(basket)       # ['Laptop', 'Mouse']

# close the program, run it again:
print(basket)       # []  -- empty again
```

WHY

Memory is wiped the moment the program ends.

Everything you have built so far was temporary.

A file lives on disk, so it is still there tomorrow.

THREE WAYS TO OPEN A FILE



"r" READ

The default. Fails if the file is missing.



"w" WRITE

Creates the file — and erases it if it already exists.



"a" APPEND

Adds to the end. Keeps what was already there.

ALWAYS USE with open()

```
# the careless way
f = open("notes.txt", "w")
f.write("hello")
f.close()           # forget this and you lose data

# the safe way
with open("notes.txt", "w") as f:
    f.write("hello")
# the file is closed here, even if the code above
crashed
```

WHY with

It closes the file for you, always.

Until a file is closed, what you wrote may still be sitting in a buffer.

One habit, zero lost data. Use it every time.

READING, TWO WAYS

```
# everything at once -- fine for small files
with open("notes.txt") as f:
    text = f.read()
print(text)

# line by line -- fine for files of any size
with open("notes.txt") as f:
    for line in f:
        print(line.strip())
```

NOTE

Looping a file gives you one line per turn.

.strip() removes the newline character at the end of each line.

Never .read() a file you cannot fit in memory.

WRITING AND APPENDING

```
rows = ["Laptop", "Mouse", "Cable"]

with open("products.txt", "w") as f:
    for name in rows:
        f.write(name + "\n")

with open("products.txt", "a") as f:
    f.write("Hub\n")
```

WATCH OUT

Opening with "w" erases the file immediately — before you write anything.

write() does not add a line break. You add \n yourself.

Use "a" when you are adding to a log.

02

CSV

The language of business data

Dr. Pejman Ebrahimi

WHAT A CSV ACTUALLY IS



PLAIN TEXT

One line per row, commas between the fields. The first line is usually the column names.



NOT EXCEL

Excel opens it, but a CSV holds no colours, no formulas and no types. Every value is text.

READING A CSV

```
import csv

with open("products.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["price"])

# each row is a dictionary -- Session 3, now from disk
```

DictReader

It reads the header line and turns every row into a dictionary.

Your list of dictionaries from Session 3, loaded from a file.

Always pass newline="" or you get blank rows on Windows.

WRITING A CSV

```python
import csv

rows = [
    {"name": "Laptop", "price": 899.0},
    {"name": "Mouse",  "price": 24.5},
]

with open("out.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "price"])
    writer.writeheader()
    writer.writerows(rows)
```

DictWriter

fieldnames fixes the column order.

writeheader() writes the first line. Forget it and the file has no columns.

writerows() takes the whole list at once.

FOUR THINGS THAT BREAK A CSV



THE COMMA

A product called “Mouse, wireless” splits into two columns



ENCODING

Accents arrive as nonsense. Pass encoding=“utf-8”



EVERY VALUE IS TEXT

“899.0” is not a number until you convert it



THE NEWLINE

Blank rows on Windows unless you pass newline=“”



JSON

The language of the web

WHAT JSON IS



ALMOST PYTHON

A dictionary written as text. Curly braces, square brackets, quotes around the keys.



THE STANDARD

Every API you will ever call speaks JSON. Part two of today depends on it.

SAVING AND LOADING JSON

```python
import json

catalogue = [
    {"name": "Laptop", "price": 899.0, "stock": 4},
    {"name": "Mouse",  "price": 24.5,  "stock": 31},
]

with open("catalogue.json", "w") as f:
    json.dump(catalogue, f, indent=2)

with open("catalogue.json") as f:
    loaded = json.load(f)

print(loaded[0]["price"])       # 899.0
```

FOUR NAMES

dump writes to a file, load reads from one.

dumps and loads do the same with a string — note the s.

indent=2 makes the file readable by a human.

Numbers stay numbers. CSV cannot do that.

CSV OR JSON?

	CSV	JSON
Shape	A flat table	Nested, any depth
Types	Everything is text	Numbers, booleans, null
Use it for	Exports and reports	APIs and settings

JSON HOLDS SHAPES CSV CANNOT

```
order = {
    "id": 1042,
    "customer": {"name": "Anna", "city": "Vaduz"},
    "items": [
        {"product": "Laptop", "qty": 1},
        {"product": "Mouse",  "qty": 2},
    ],
    "paid": True,
}

print(order["customer"]["city"])     # Vaduz
print(order["items"][1]["product"]) # Mouse
```

READ IT LEFT TO RIGHT

A dictionary inside a dictionary. A list of dictionaries inside that.

Each bracket takes you one level deeper.

Try writing this as a CSV — you cannot, and that is the point.

04

ERRORS

Programs that survive bad input

THE FILE THAT IS NOT THERE

```
with open("catalogue.json") as f:
    data = json.load(f)

# FileNotFoundError: [Errno 2] No such file
# or directory: 'catalogue.json'

# The program stops. The user sees a red wall.
# On a first run, the file never exists.
```

THINK AHEAD

This is not a bug in your code. It is a situation your code must expect.

First run, deleted file, wrong folder, no permission — all normal.

Handling it is the difference between a script and a program.

try / except, PROPERLY

```
import json

try:
    with open("catalogue.json") as f:
        catalogue = json.load(f)
except FileNotFoundError:
    print("No file yet -- starting empty.")
    catalogue = []
else:
    print(f"Loaded {len(catalogue)} products.")
finally:
    print("Ready.")
```

FOUR WORDS

try — the risky part, kept as short as you can.

except — what to do about one named problem.

else — runs only when nothing went wrong.

finally — runs either way. Cleanup goes here.

THREE ERRORS YOU WILL MEET TODAY



FileNotFoundError

The path is wrong, or it is the first run



JSONDecodeError

The file exists but is empty or half-written



KeyError

The row has no such field. Use .get() instead

NEVER CATCH AN ERROR YOU CANNOT HANDLE

05

THE BUILD

A catalogue that survives a restart

THE BUILD, IN FOUR STEPS



LOAD

Read catalogue.json, or start with an empty list if it is missing



SHOW

Print the products with a number beside each one



CHANGE

Let the user add a product or change a price



SAVE

Write the whole list back, so the next run sees it

LOAD, CHANGE, SAVE

```
import json

FILE = "catalogue.json"

def load():
    try:
        with open(FILE) as f:
            return json.load(f)
    except FileNotFoundError:
        return []

def save(rows):
    with open(FILE, "w") as f:
        json.dump(rows, f, indent=2)

catalogue = load()
catalogue.append({"name": "Hub", "price": 149.0})
save(catalogue)
```

PATHS THAT WORK EVERYWHERE

```python
from pathlib import Path

# this breaks on the other operating system
bad = "data\\catalogue.json"

# this works on all of them
good = Path("data") / "catalogue.json"

if good.exists():
    print(good.read_text()[:40])
```

WHY IT MATTERS

Windows writes a backslash, macOS and Linux write a forward slash.

Path joins them with / and gets it right on every machine.

.exists() asks before you open — handy, but try/except is still safer.

HACKERRANK — TAKE HOME

PROBLEM	STREAM	WHAT IT TEACHES
Exceptions	A	try / except on real input
collections.Counter()	A	Dictionaries doing real work
Two Strings	B	Set and dictionary membership

HOMEWORK

No homework for this session.

THANK YOU

Dr. Pejman Ebrahimi
pejman.ebrahimi@uni.li

University of Liechtenstein